

CVIS Hands-on Training on Development of Machine Learning and Deep Learning Models Summer 2025

Facilitator
Nafiz Intiaz Rafin
Adjunct Lecturer



Compute Vision and Intelligent Systems Lab
Dept. Of Computer Science and Engineering
School of Data and Sciences, BRAC University

Core Machine Learning Algorithms



Introduction

In this session, we will cover some of the most important classical machine learning algorithms that form the foundation of modern data science and AI systems. These include both supervised and unsupervised techniques, as well as dimensionality reduction methods. We will not only explore the theoretical underpinnings of each algorithm but also provide practical Python code examples to demonstrate how they work in real-world scenarios.



By the end of this lecture, participants should be able to:

- Understand the concepts and intuition behind each ML algorithm.
- Identify appropriate use cases for different algorithms.
- Implement these algorithms using popular Python libraries like Scikit-learn.



1. Support vector machines
2. Decision trees and forests
3. Clustering, K-means, and Gaussian mixture models
4. Principal component analysis



1. Support Vector Machines (SVM)

Definition:

Support Vector Machine (SVM) is a **supervised learning algorithm** used for classification and regression problems. It finds the **optimal hyperplane** that best separates the data into classes with the **maximum margin**.



✓ Key Concepts:

- **Hyperplane:** A decision boundary in a high-dimensional space that separates classes.
- **Margin:** Distance between the hyperplane and the nearest data points (support vectors).
- **Support Vectors:** Data points that lie closest to the hyperplane and influence its position.
- **Kernel Trick:** Transforms input data into a higher-dimensional space to handle non-linear separation (e.g., Polynomial, RBF kernels).



✓ Advantages:

- Effective in high-dimensional spaces
- Memory efficient (uses a subset of training points)
- Versatile with different kernel functions

✓ Limitations:

- Not suitable for very large datasets (training time)
- Difficult to choose the right kernel



✓ Python Code (Scikit-learn):

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X, y = datasets.load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

model = SVC(kernel='rbf')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```



2. Decision Trees and Random Forests

✓ Decision Trees:

A **decision tree** is a supervised learning algorithm that splits data based on feature values, forming a tree-like model of decisions.

✓ Random Forest:

An **ensemble learning method** using multiple decision trees trained on random subsets (bagging). Final prediction is based on majority vote or averaging.



✓ Key Concepts:

- **Nodes:** Internal decision points based on feature values
- **Leaves:** Final outcomes (classes)
- **Gini Impurity / Entropy:** Measures of impurity used to split nodes
- **Overfitting:** Deep trees may memorize data
- **Pruning:** Reduces tree size to avoid overfitting

✓ Advantages:

- Handles both classification and regression
- Works with non-linear relationships
- Random forests improve robustness



✓ Python Code:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Decision Tree
dt = DecisionTreeClassifier(max_depth=3)
dt.fit(X_train, y_train)

# Random Forest
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)

print("Decision Tree Accuracy:", dt.score(X_test, y_test))
print("Random Forest Accuracy:", rf.score(X_test, y_test))
```



3. Clustering: K-Means and Gaussian Mixture Model (GMM)

Clustering Overview:

Clustering is an **unsupervised learning** task to group data points into clusters such that data points in the same group are more similar to each other.



✓ K-Means Clustering:

- **Objective:** Minimize intra-cluster distance
- **Steps:**
 1. Initialize k centroids
 2. Assign each point to the nearest centroid
 3. Update centroids as mean of assigned points
 4. Repeat until convergence
- **Drawbacks:** Sensitive to initialization, assumes spherical clusters



Python Code:

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=3, random_state=42)  
kmeans.fit(X)
```

```
print("Cluster Centers:\n", kmeans.cluster_centers_)  
print("Labels:", kmeans.labels_)
```



✓ Gaussian Mixture Model (GMM):

- **Probabilistic clustering** using a mixture of Gaussians
- Soft assignment: each point has a probability of belonging to a cluster
- Uses **Expectation-Maximization (EM)** algorithm
- Suitable for elliptical clusters



Python Code:

```
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=3, random_state=42)
gmm.fit(X)

print("Means:\n", gmm.means_)
print("Covariances:\n", gmm.covariances_)
```



4. Principal Component Analysis (PCA)

Definition:

PCA is a **dimensionality reduction** technique that transforms features into a set of **linearly uncorrelated components** ordered by variance.



✓ Intuition:

- Find new axes (principal components) where the data has the most variance
- Project data into fewer dimensions while preserving structure

✓ Steps:

1. Center the data (subtract the mean)
2. Compute the covariance matrix
3. Find eigenvectors and eigenvalues
4. Project data on top-k eigenvectors



Applications:

- Visualization (2D/3D)
- Noise filtering
- Speed up ML algorithms



Python Code:

```
from sklearn.decomposition import PCA  
  
pca = PCA(n_components=2)  
  
X_pca = pca.fit_transform(X)  
  
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```



Recommended References & Online Resources

General Machine Learning

- Scikit-learn Documentation:
https://scikit-learn.org/stable/user_guide.html
- Google Machine Learning Crash Course:
<https://developers.google.com/machine-learning/crash-course>
- Towards Data Science ML Tutorials:
<https://towardsdatascience.com/tagged/machine-learning>
- StatQuest with Josh Starmer (YouTube):
<https://www.youtube.com/user/joshstarmer>



Support Vector Machines (SVM)

- Scikit-learn SVM: <https://scikit-learn.org/stable/modules/svm.html>
- Visual Explanation of SVM:
<https://www.analyticsvidhya.com/blog/2017/09/understaing-support-vector-machine-example-code/>

Decision Trees and Random Forests

- Decision Trees in Sklearn: <https://scikit-learn.org/stable/modules/tree.html>
- Random Forest Explained:
<https://towardsdatascience.com/understanding-random-forest-58381e0602d2>
- Interpreting Trees with Graphviz:
<https://scikit-learn.org/stable/modules/tree.html#visualization>



Clustering (K-Means, GMM)

- Clustering Overview in Sklearn: <https://scikit-learn.org/stable/modules/clustering.html>
- K-Means Intuition and Examples:
<https://www.analyticsvidhya.com/blog/2019/08/comprehensive-guide-k-means-clustering/>
- Gaussian Mixture Models: <https://scikit-learn.org/stable/modules/mixture.html>
- EM Algorithm Simplified:
<https://towardsdatascience.com/expectation-maximization-em-algorithm-explained-7c715068dcd9>

Principal Component Analysis (PCA)

- PCA User Guide in Sklearn: <https://scikit-learn.org/stable/modules/decomposition.html#pca>
- PCA Explained Visually: <https://setosa.io/ev/principal-component-analysis/>
- Step-by-step PCA from scratch:
<https://towardsdatascience.com/principal-component-analysis-for-dummies-224b0e16a492>